Copyright All rights reserved

18. 디버깅과 오류 추적
(Debugging and Trace Error)

Heesung Oh
https://github.com/3dapi

18.1 컴파일 오류, 링크 오류, 런타임 오류

Heesung Oh
Copyright All rights reserved

- 오류 발생 단계
  - 전처리 오류
  - 컴파일 오류
  - 링크 오류
  - 런타임 오류

→ 단계별 원인 분석 필요

18.1 전처리 오류

Heesung Oh
Copyright All rights reserved

- include 구조 문제

- 매크로 확장
  - 예)

```
#define SIZE 100;
int arr[SIZE];

// 전처리 후:
int arr[100;];
```

→ 잘못된 코드 생성 → 이후 컴파일 단계에서 오류 발생

18.1 컴파일 오류

Heesung Oh
Copyright All rights reserved

- 문법 또는 타입 오류
  - 타입 불일치
  - 세미콜론 누락
  - 미 선언 식별자

```
int main(void)
{
    int a = "text";
}
```

→ 실행 파일 생성 실패

18.1 링크 오류

Heesung Oh
Copyright All rights reserved

- 심볼 결합 실패
- 선언만 있고 정의 없음

```
int add(int a, int b);    // 선언만 있음
extern int g_value;       // extern 선언만
있음

int main(void)
{
    return add(1,2);
}

// 오류 예:
unresolved external symbol add
// 또는
multiple definition of value
```

→ 실행 파일 생성 실패

18.1 런타임 오류

Heesung Oh
Copyright All rights reserved

- 실행 중 발생
  - 빌드는 성공
  - 실행 중 비정상 동작
  - 추적이 가장 어려움
  - 특징
    - 정의되지 않은 동작
    - 즉시 오류가 아닐 수 있음

```
int* p = NULL;
*p = 10;
```

```
int arr[5];
arr[5] = 10;
```

```
free(p);
*p = 10;
```

```
printf("%d", "text");
```

- 예)
  - NULL 포인터 접근, 배열 범위 초과, 해제 후 접근
  - 타입 불일치 → 잘못된 메모리 해석

18.2 printf 디버깅

Heesung Oh
Copyright All rights reserved

- 값 출력으로 흐름 확인
- 용도
  - 값, 분기, 실행 흐름 추적

- 예) 디버그 전용 로그
  - Release 에서는 제거됨

```
#ifdef _DEBUG
  #define LOG(msg) printf("[DEBUG] %s\n", msg)
#else
  #define LOG(msg)
#endif
```

Heesung Oh
Copyright All rights reserved

- 가변 인자
  - va_list, va_start, vprintf, va_end

```
// 기본 구조
#include <stdio.h>
#include <stdarg.h>

void log_print(const char* fmt, ...)
{
    va_list args;
    va_start(args, fmt);
    vprintf(fmt, args);
    va_end(args);
}

// 사용:
log_print("value = %d, name = %s\n", 10, "test");
```

18.3 위치 정보 로그

Heesung Oh
Copyright All rights reserved

- __FILE__, __LINE__, __func__ → 로그에 위치 정보 자동 포함

```
// 위치 정보 자동 포함
#define LOG(fmt, ...) \
    log_internal(__FILE__, __LINE__, \
                 __func__, fmt, __VA_ARGS__)

void log_internal(const char* file, int line,
                  const char* func, const char* fmt, ...)
{
    printf("[%s:%d:%s] ", file, line, func);

    va_list args;
    va_start(args, fmt);
    vprintf(fmt, args);
    va_end(args);
}
```

18.4 디버거 사용 기초

Heesung Oh
Copyright All rights reserved

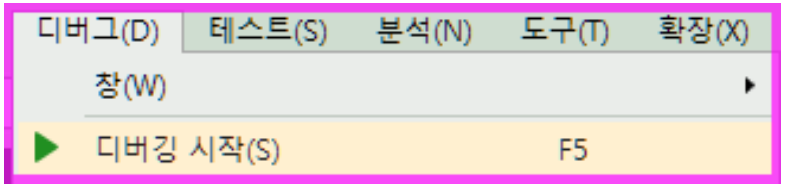
- 디버거
  - 프로그램 실행 제어 도구
  - 실행 위치 확인
  - 변수 상태 확인
  - 상태 기반 분석

- MSVC 디버거
  - 디버깅 시작
  - Breakpoint / Data Breakpoint 설정
  - Step into / Step over /Step out
  - Call stack / Watch / Memory 창

18.4 디버깅 시작 / Breakpoint / Step

Heesung Oh
Copyright All rights reserved

- 디버깅 시작
  - F5 → 실행
  - Shift+F5 → 종료



- Break point
  - 코드 실행 중지 지점
  - 설정 / 해제 F9 토글

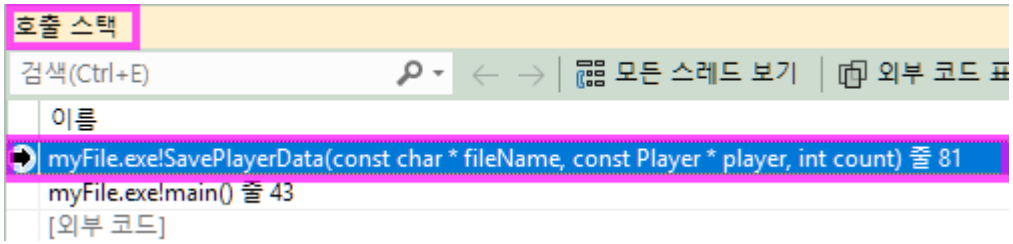
```
80      // 플레이어 숫자 저장.
81      fwrite(&count, sizeof(int), 1, fp);
```

- Step 실행
  - F11 → Step Into
  - F10 → Step Over
  - Shift+F11 → Step Out

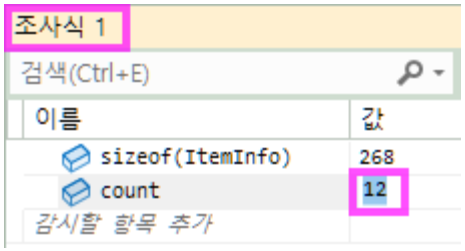
18.4 디버깅 윈도우

Heesung Oh
Copyright All rights reserved

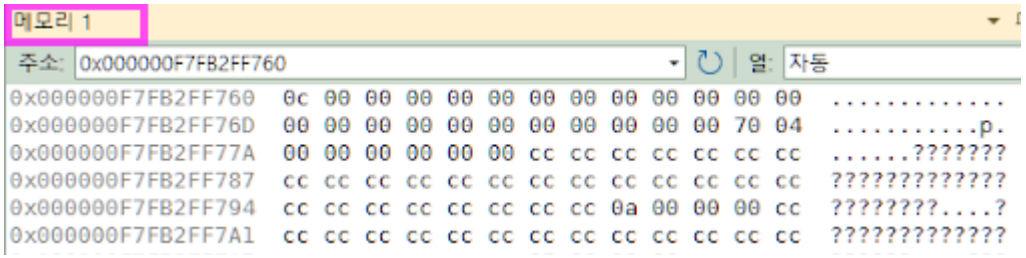
- 호출 스택 (Call stack)
  - 함수 호출 흐름 확인
  - 호출 경로 분석
  - 의도된 흐름 검증



- 조사식 (Watch)
  - 변수 값 실시간 확인
  - 표현식 등록 가능
  - 값 변경 가능



- 메모리
  - 메모리 직접 확인
  - HEX 형태 표시



18.4 디버깅 윈도우

Heesung Oh
Copyright All rights reserved

- Disassembly
  - 어셈블리 코드 확인
  - 크래시 위치 확인

```
    // 플레이어 숫자 저장.
    fwrite(&count, sizeof(int), 1, fp);
00007FF751C21AAE  mov         r9,qword ptr [fp]
00007FF751C21AB2  mov         r8d,1
00007FF751C21AB8  mov         edx,4
00007FF751C21ABD  lea         rcx,[count]
00007FF751C21AC4  call        qword ptr [__imp_fwrite (07FF751C32318h)]
```

- 데이터 중단점
  - 변수 값 변경 시 중단
  - 값 변경 위치 추적
  - 원인 분석에 효과적

새 데이터 중단점

지정된 주소에서부터 지정된 바이트 수가 수정된 경우 실행을 중단합니다.

주소: '0x000000F7FB2FF760' (4 바이트)

☑ 조건

조건식 | 참인 경우 | 예: x == 5 | 취소

조건 추가

☐ 작업

☐ 적중되면 중단점 사용 안 함

☐ 다음 중단점이 적중된 경우에만 사용하도록 설정합니다.

18.5 Windows 메모리 디버깅 기초

Heesung Oh
Copyright All rights reserved

- CRT Debug Heap
  - 메모리 누수 추적
  - Debug 빌드에서만 동작
  - 자동 리포트 가능
- 누수 덤프
  - _CrtDumpMemoryLeaks()
  - 특정 시점 검사 가능
- 누수 검사
  - _CrtSetDbgFlag
  - 할당 추적
  - 종료 시 누수 출력
- BreakAlloc
  - _CrtSetBreakAlloc
  - 특정 할당 시점 중단
    → 누수 위치 추적

18.5 Windows 메모리 디버깅 기초

Heesung Oh
Copyright All rights reserved

- 헤더 포함 순서 (매우 중요)

```
#define _CRTDBG_MAP_ALLOC
#include <crtdbg.h>
#include <stdio.h>
```

- 종료 시 자동 누수 리포트 활성화

```
int main(void) {
  int flags = _CrtSetDbgFlag(_CRTDBG_REPORT_FLAG);
  flags |= _CRTDBG_ALLOC_MEM_DF;   // 메모리 할당 추적
  flags |= _CRTDBG_LEAK_CHECK_DF;  // 종료 시 자동 누수 검사
  _CrtSetDbgFlag(flags);
```

- 할당 번호로 누수 추적하기 (_CrtSetBreakAlloc)

```
Detected memory leaks!
{12345} normal block at 0x0000021F...
// 이 번호를 _CrtSetBreakAlloc()에 지정하면,
// 해당 메모리가 할당되는 순간 디버거가 자동으로 멈춤

int main(void)
{
    // 예: 누수 리포트에서 검출된 {12345} 를 지정
    _CrtSetBreakAlloc(12345);
```

학습 정리

Heesung Oh
Copyright All rights reserved

- 디버깅 단계 구분
  - 전처리 / 컴파일 / 링크 / 실행
- 디버깅 로그 출력, 디버거 활용
- 메모리 문제 중요→ 상태를 이해